

Rockchip AOV 开发指南

文件标识: RK-KF-YF-A17

发布版本: V1.1.2

日期: 2025-09-29

文件密级: ☐绝密 ☐秘密 ☐内部资料 ☒公开

免责声明

本文档按“现状”提供, 瑞芯微电子股份有限公司 (“本公司”, 下同) 不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因, 本文档将可能在未经任何通知的情况下, 不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标, 归本公司所有。

本文档可能提及的其他所有注册商标或商标, 由其各自所有者所有。

版权所有 © 2025 瑞芯微电子股份有限公司

超越合理使用范畴, 非经本公司书面许可, 任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部, 并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址: 福建省福州市铜盘路软件园A区18号

网址: www.rock-chips.com

客户服务电话: +86-4007-700-590

客户服务传真: +86-591-83951833

客户服务邮箱: fae@rock-chips.com

前言

概述

本文主要介绍AOV(Always On Video)的基本功能和开发指南。

产品版本

芯片名称	内核版本
RV1106	Linux 5.10
RV1106B	Linux 5.10
RV1126B	Linux 6.10

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	YYZ, HL, HKH, CWW	2023-12-12	初始版本
V1.0.1	XY, MLC	2023-12-25	添加WiFi/4G、摄像头驱动说明
V1.0.2	ZY	2024-01-10	添加双目说明
V1.0.3	HKH	2024-02-19	添加休眠状态时钟源说明
V1.0.4	HKH	2024-04-27	更新调试说明
V1.1.0	ZY	2024-10-16	1. 添加EMMC说明 2. 添加双目非融合拼接说明 3. 添加usb、sdio驱动说明 4. 添加RV1106B功耗数据
V1.1.1	CWW	2025-03-08	更新 AOV 工作流程说明
V1.1.2	SL	2025-09-28	1.更新了AOA 模块说明 2. 添加了1126B 的支持

目录

Rockchip AOV 开发指南

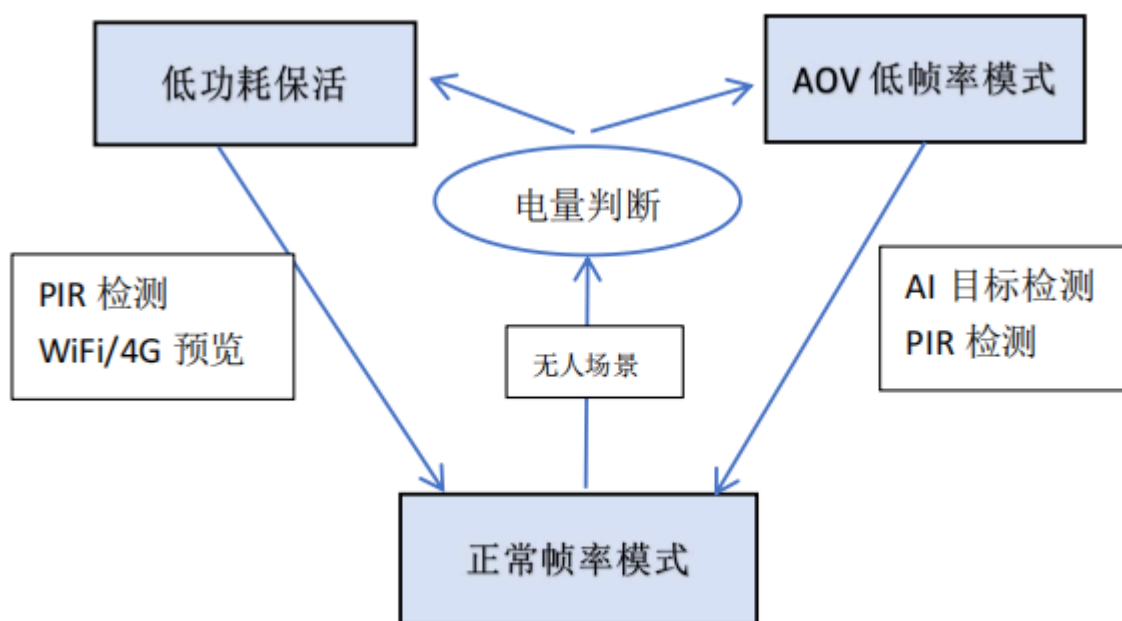
1. AOV简介
 - 1.1 场景说明
 - 1.2 AOV 工作流程说明
2. AOA 简介
 - 2.1 音频数据流
3. 硬件说明
 - 3.1 硬件接口说明
 - 3.1.1 RV1106接口功能
 - 3.1.2 RV1106B接口功能
 - 3.1.3 RV1126B 接口功能
4. 开发说明
 - 4.1 AOV状态切换
 - 4.1.1 设置AOV定时休眠时间
 - 4.1.2 从正常模式进入AOV
 - 4.1.3 从AOV模式进入正常模式
 - 4.1.4 AOV模式
 - 4.2 AOA 接口说明
 - 4.2.1 初始化 AOA 模块
 - 4.2.2 销毁 AOA 模块
 - 4.2.3 启动 AOA 模块
 - 4.2.4 停止 AOA 模块
 - 4.2.5 音频数字放大控制
 - 4.2.6 获取音频数据
 - 4.2.7 音频帧缓存
 - 4.2.8 从缓存中读出数据送往编码器
 - 4.2.9 aoa_configs
 - 4.2.10 aoa_results
 - 4.3 AOV配置
 - 4.3.1 系统配置
 - 4.3.1.1 Kernel
 - 4.3.1.2 BoardConfig
 - 4.3.2 按键唤醒配置
 - 4.3.3 摄像头驱动
 - 4.3.3.1 Standby模式
 - 4.3.3.2 掉电模式
 - 4.3.3.3 补光灯控制
 - 4.3.4 WiFi/4G开发说明
 - 4.3.5 休眠状态时钟源
 - 4.3.6 基于eMMC的AOV开发说明
 - 4.3.6.1 板级配置
 - 4.3.6.2 应用流程
 - 4.4 AOA配置
 - 4.4.1 板级配置
 - 4.4.2 应用流程
5. 调试说明
 - 5.1 Demo说明
 - 5.1.1 ISP 取流测试
 - 5.1.2 主码流编码测试
 - 5.1.3 人形检测测试

- 5.1.4 AIISP测试
 - 5.1.5 双摄ISP取流测试
 - 5.1.6 双摄主码流编码测试
 - 5.1.7 双摄人形检测测试
 - 5.1.8 双摄上下非融合拼接测试
- 5.2 休眠唤醒耗时排查
 - 5.2.1 AOV耗时确认
 - 5.2.2 软件耗时排查
- 5.3 排查音频卡顿掉帧
- 5.4 排查视频不出流问题
- 6. 典型功耗数据
 - 6.1 基础休眠功耗
 - 6.1.1 RV1106
 - 6.1.2 RV1106B
 - 6.2 RV1106 SC200AI数据
 - 6.2.1 编码一帧+人形检测
 - 6.2.2 AIISP+编码一帧+人形检测
 - 6.3 RV1106B SC450AI数据
 - 6.3.1 编码一帧 + 人形检测
 - 6.4 RV1106B 双目SC200AI数据
 - 6.4.1 非融合拼接+编码一帧+人形检测
- 7. 常见问题
 - 7.1 应用层时间怎么加上休眠时间
 - 7.2 如何获取唤醒源
 - 7.3 为何AOV SDK开机后SD 卡无法识别
 - 7.4 如何解决AOV过程中，SD卡或者4G传输数据时短暂没数据
 - 7.5 如何更低功耗控制红外灯

1. AOV简介

AOV(Always On Video)基于休眠唤醒实现的24小时低帧率持续录像，有事件时也可以切回正常帧率模式，如AI检测、PIR触发、WIFI唤醒等。

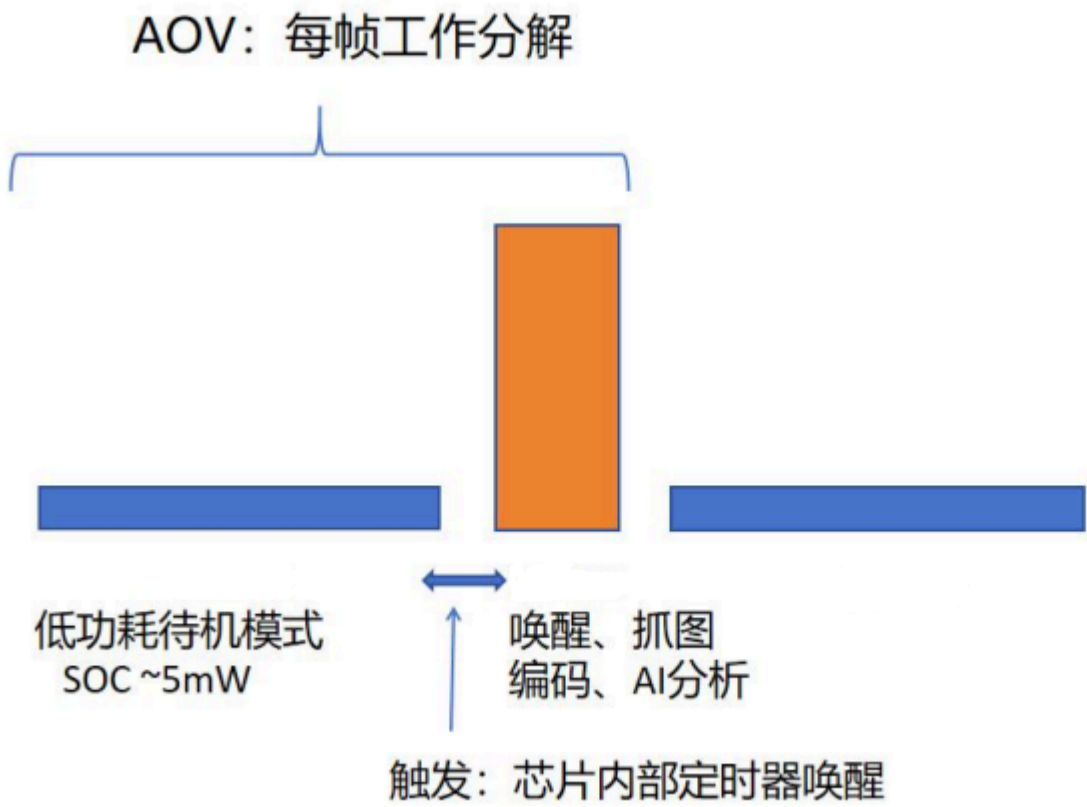
1.1 场景说明



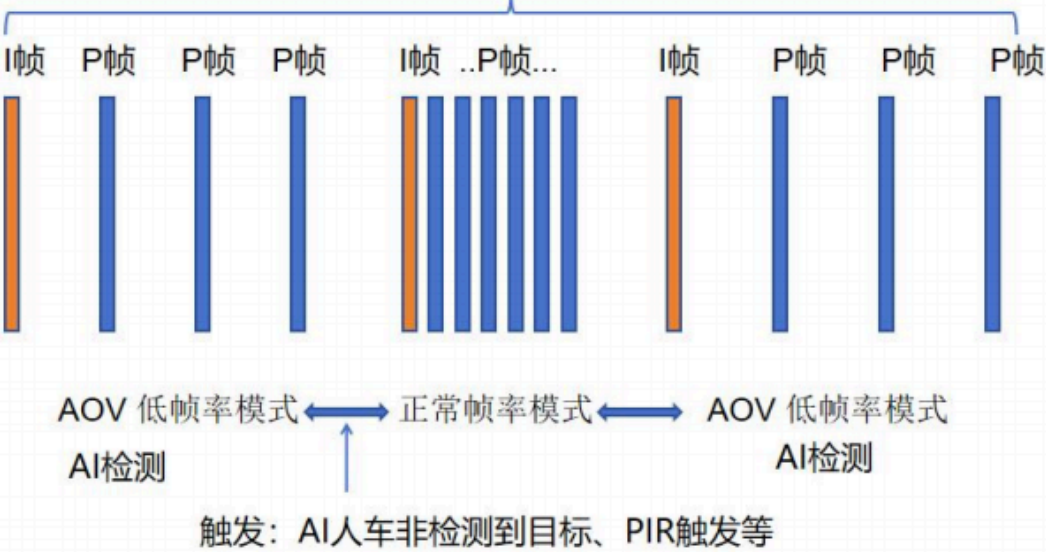
三个场景：

- 正常帧率模式：远程预览、PIR 事件、人形检测事件时进入。需要录像、联网等功能；
- AOV 低帧率模式：空闲时，电量足，SOC 低帧率主码流录像、WiFi/4G 保活；
- 低功耗保活模式：空闲时，电量较低时，SOC 下电、WiFi/4G 保活。

1.2 AOV 工作流程说明



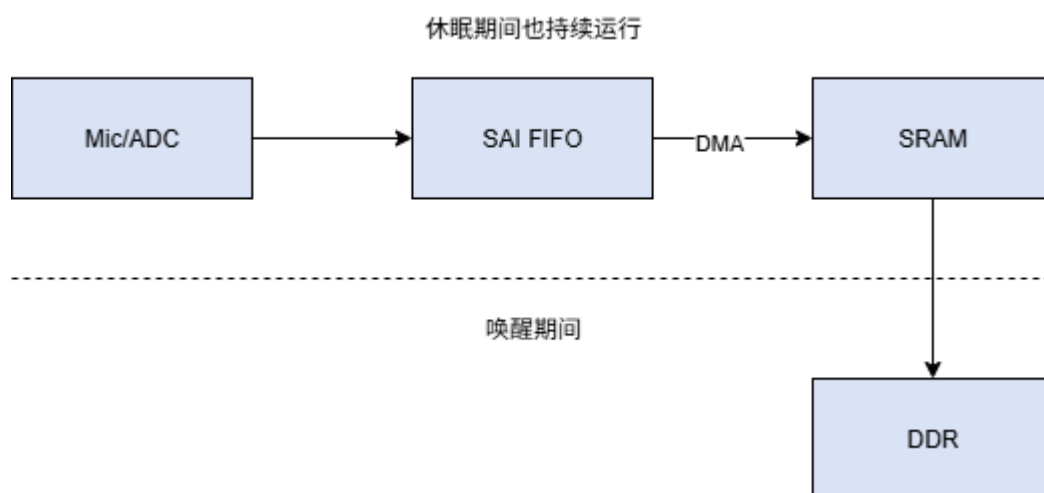
AOV: 低帧率持续录像，典型场景：1秒1帧
H264/H265编码



2. AOA 简介

AOA (ALway On Audio) 休眠期间通过SRAM 缓存音频数据，实现24小时持续录音。

2.1 音频数据流



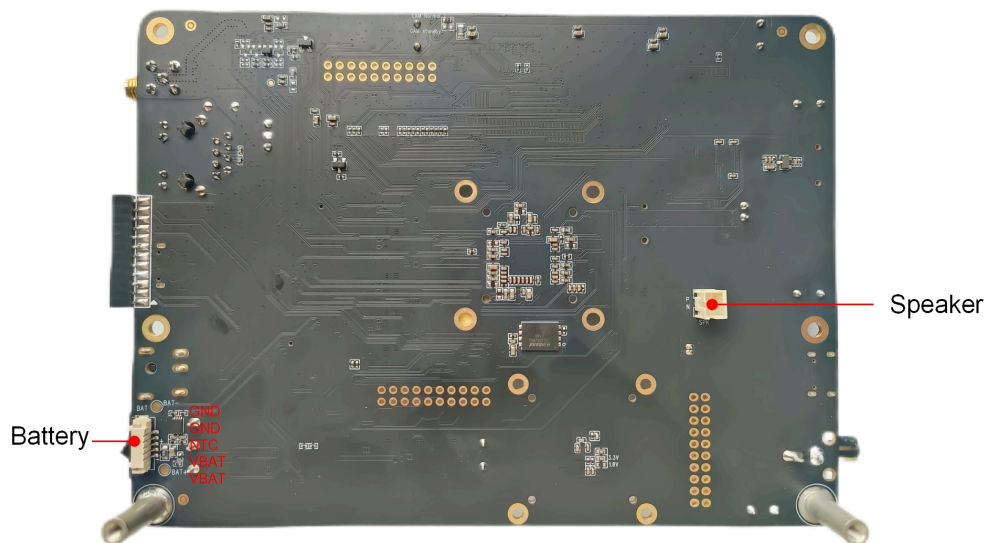
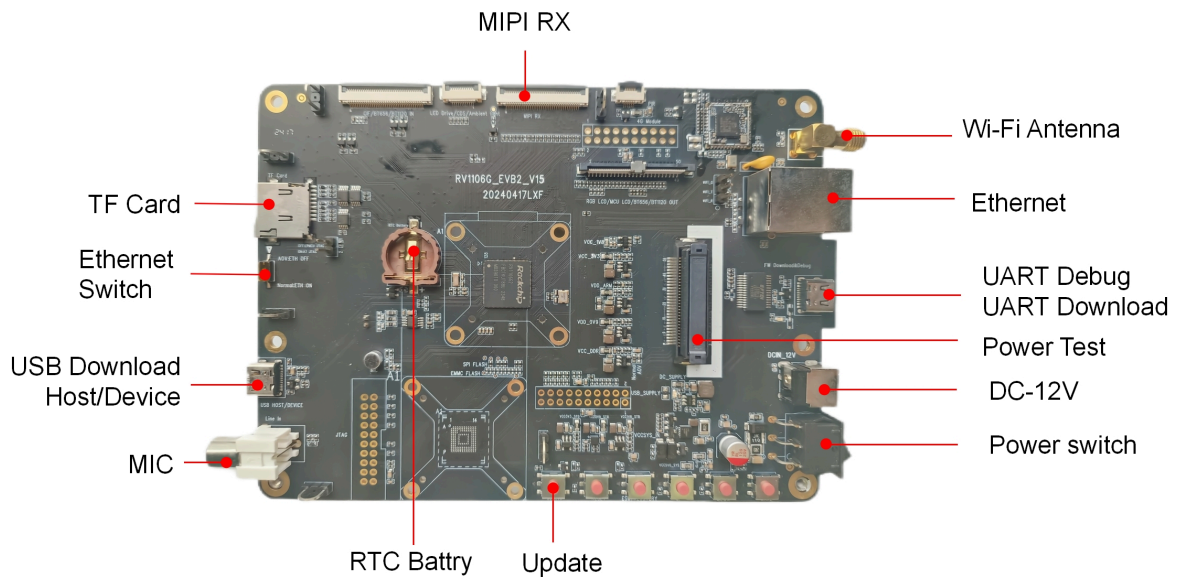
内存限制

- 休眠期间数据暂存于内置64KB SRAM，可外挂更大SRAM延长录音时间
- 每帧512Byte，可存约113帧
- 1秒约62.5帧，最长休眠1.8秒不丢数据

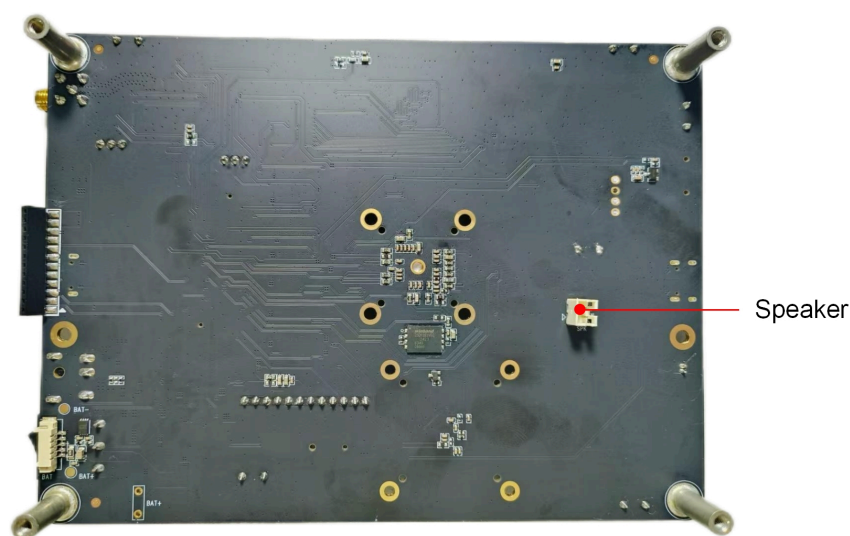
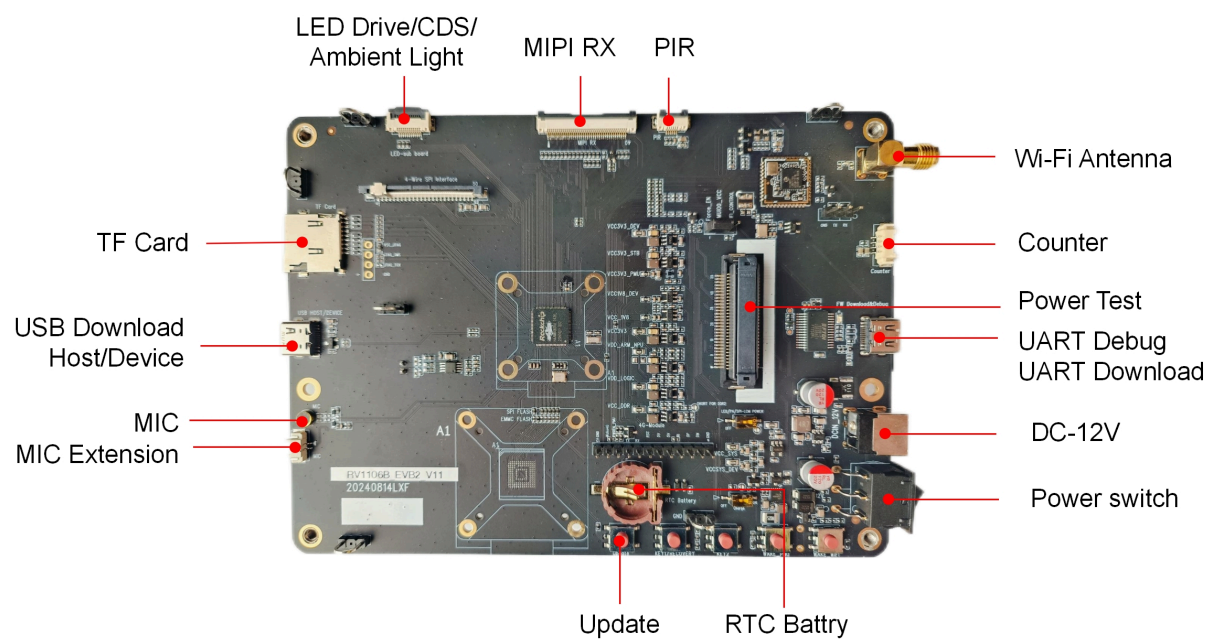
3. 硬件说明

3.1 硬件接口说明

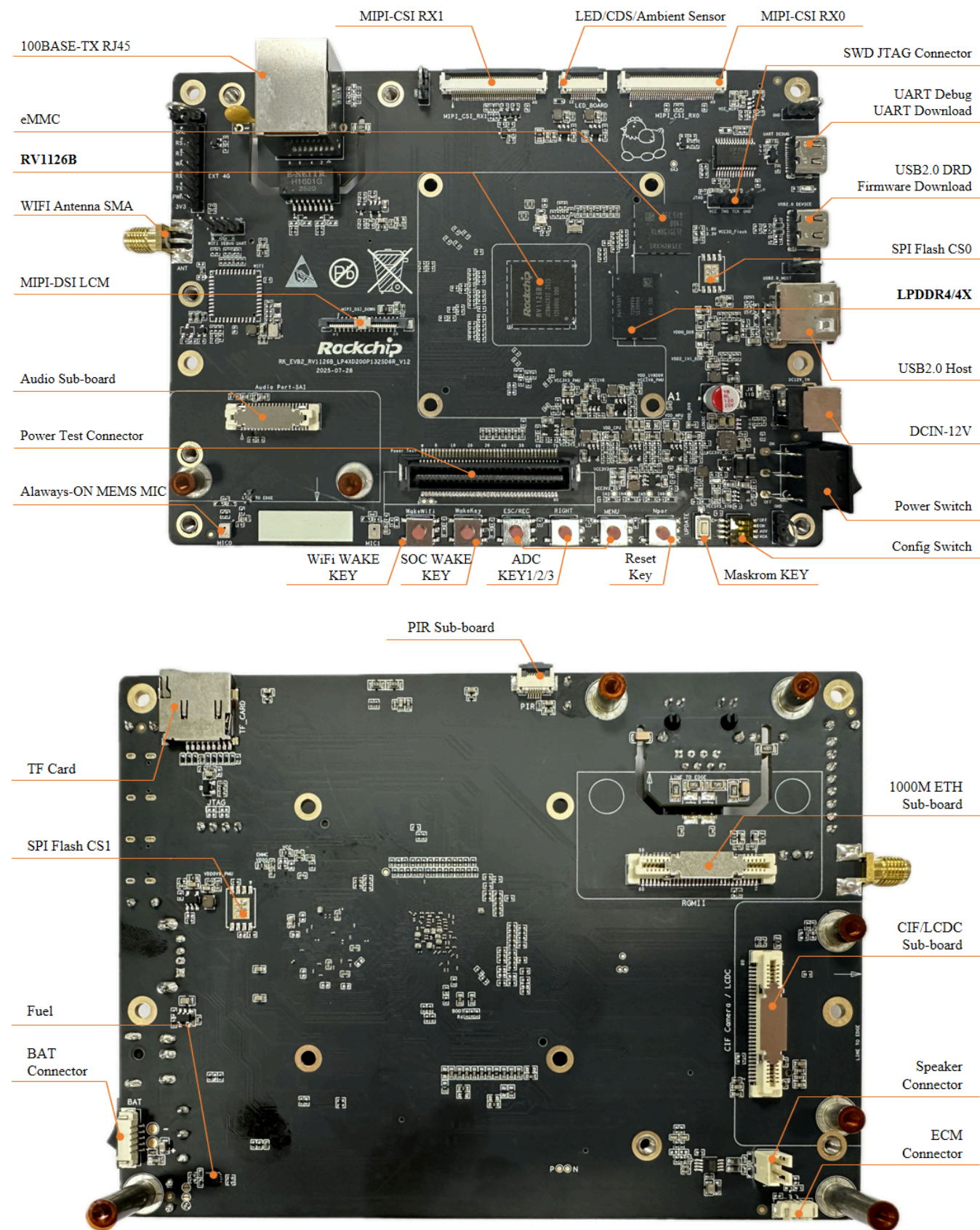
3.1.1 RV1106接口功能



3.1.2 RV1106B接口功能



3.1.3 RV1126B 接口功能



4. 开发说明

4.1 AOV状态切换

4.1.1 设置AOV定时休眠时间

最大支持37小时，单位ms，详细可以参考3.2.5章节

以下接口二选一，如果需要固定休眠1s则使用第二个，如果需要出帧间隔1s则使用第二个

```
// 设置休眠时间
int SAMPLE_COMM_AOV_SetSuspendTime(int wakeup_suspend_time);
// 设置出帧间隔时间
int SAMPLE_COMM_AOV_SetPeriodTime(int wakeup_period_time);
```

4.1.2 从正常模式进入AOV

从正常帧率进入AOV模式，需要除AOV录像外的媒体数据流和卸载外部设备来节省功耗，如下：

1. 从正常帧率模式进入AOV单帧模式，需要将系统剩余的帧取完，防止残留帧影响休眠唤醒

```
SAMPLE_COMM_ISP_SingleFrame(0);
// drop frame
VIDEO_FRAME_INFO_S stViFrame_tmp;
while (RK_MPI_VI_GetChnFrame(ctx->s32DevId, ctx->s32ChnId, &stViFrame_tmp, 1000) ==
      RK_SUCCESS) {
    RK_MPI_VI_ReleaseChnFrame(ctx->s32DevId, ctx->s32ChnId, &stViFrame_tmp);
}
```

2. 关闭子码流、音频等AOV模式下不需要的媒体数据流

```
# 暂停子码流
RK_MPI_VI_PauseChn(ctx->vi[2].u32PipeId, ctx->vi[2].s32ChnId);
# 关闭音频
TBD
```

3. 卸载外设，节省休眠时和唤醒时功耗

SD卡：

```
int SAMPLE_COMM_AOV_UnbindSdcard(); // 卸载SD卡
```

以太网：

```
int SAMPLE_COMM_AOV_UnbindEthernet(); // 卸载以太网
```

音频驱动：

```
int SAMPLE_COMM_AOV_UnbindSoundcard(); // 卸载音频
```

USB驱动：

```
int SAMPLE_COMM_AOV_UnbindUsb(); // 卸载USB驱动
```

SDIO驱动：

```
int SAMPLE_COMM_AOV_UnbindSDIO(); // 卸载SDIO驱动
```

4. 进入AOV

```
void SAMPLE_COMM_AOV_EnterSleep();
```

4.1.3 从AOV模式进入正常模式

主要是做媒体数据流和外设的恢复，具体参考如下

1. 恢复多帧模式

```
RK_S32 SAMPLE_COMM_ISP_MultiFrame(int cam_id);
```

2. 恢复子码流、音频等媒体数据流

```
# 恢复子码流
RK_MPI_VI_ResumeChn(ctx->vi[2].u32PipeId, ctx->vi[2].s32ChnId);
# 初始化AI/AO
TBD
```

3. 恢复外设

SD卡：

```
int SAMPLE_COMM_AOV_BindSdcard(); // 挂载SD卡
```

以太网：

```
int SAMPLE_COMM_AOV_BindEthernet(); // 挂载以太网
```

音频驱动：

```
int SAMPLE_COMM_AOV_BindSoundcard(); // 挂载音频驱动
```

USB驱动

```
int SAMPLE_COMM_AOV_BindUsb();
```

SDIO驱动

```
int SAMPLE_COMM_AOV_BindSDIO(); // 挂载SDIO驱动
```

4.1.4 AOV模式

AOV 模式可以将码流存储，等到累计到一定的值，才将码流一次性写到SD卡或者通过WIFI 传输出去，以下为SD卡写卡参考：

```
SAMPLE_COMM_AOV_CopyStreamToSdcard
```

取完码流或者做完人形检测之后立即进入AOV，如果想做更低的功耗可以先将YUV取出，待休眠唤醒之后再编码+IVS+人形检测：

```
SAMPLE_COMM_AOV_EnterSleep();
```

产品如果关注亮度收敛时间，可以使用以下流程进行亮度快速收敛

```
// 1. 检测是否大变化
RK_BOOL SAMPLE_COMM_ISP_IsExpBigChange(int cam_id); // 环境亮度变化大，返回TRUE
// 2. 大变化则切换到连续帧模式
// 3. 连续帧模式下判断是否收敛
RK_BOOL SAMPLE_COMM_ISP_IsExpConverge(int cam_id); // 亮度收敛之后，返回TRUE
// 4. 收敛完成后,切换为AOV模式
```

4.2 AOA 接口说明

4.2.1 初始化 AOA 模块

【语法】

```
void *aoa_create(struct aoa\_configs *config);
```

【参数】

参数名	描述	输入/输出
config	AOA 配置	输入

【返回值】

返回值	描述
void*	AOA 句柄，失败返回 NULL

4.2.2 销毁 AOA 模块

【语法】

```
int aoa_destroy(void *hdl);
```

【参数】

参数名	描述	输入/输出
hdl	AOA 句柄	输入

【返回值】

返回值	描述
int	0 表示成功，其他值表示失败

4.2.3 启动 AOA 模块

【语法】

```
int aoa_start(void *hdl);
```

【参数】

参数名	描述	输入/输出
hdl	AOA 句柄	输入

【返回值】

返回值	描述
int	0 表示成功，其他值表示失败

4.2.4 停止 AOA 模块

【语法】

```
int aoa_stop(void *hdl);
```

【参数】

参数名	描述	输入/输出
hdl	AOA 句柄	输入

【返回值】

返回值	描述
int	0 表示成功，其他值表示失败

4.2.5 音频数字放大控制

【语法】

```
int aoa_control(void *hdl, int cmd, int volume);
```

【参数】

参数名	描述	输入/输出
hdl	AOA 句柄	输入
cmd	控制命令（如 AOA_CMD_VOLUME_SET）	输入
volume	音量数值（如 0-100）	输入

【返回值】

返回值	描述
int	0 表示成功，其他值表示失败

4.2.6 获取音频数据

【语法】

```
int aoa_read(void *hdl, void *buf, int size, struct aoa\_results *res, int timeout_ms);
```

【参数】

参数名	描述	输入/输出
hdl	AOA 句柄	输入
buf	数据缓冲区指针	输出
size	读取数据大小	输入
res	结果结构体指针	输出
timeout_ms	超时时间（毫秒）	输入

【返回值】

返回值	描述
int	实际读取的数据字节数，负值表示失败

4.2.7 音频帧缓存

【语法】

int cache_audio_frm(SAMPLE_AENC_CTX_S aenc, MB_BLK blk, size_t size, RK_U64 pts, RK_U32 seq);

【参数】

参数名	描述	输入/输出
aenc	音频编码句柄	输入
blk	音频帧数据块	输入
size	帧大小	输入
pts	帧时间戳	输入
seq	帧序号	输入

【返回值】

返回值	描述
int	0 表示成功，其他值表示失败

4.2.8 从缓存中读出数据送往编码器

【语法】

```
int SAMPLE_COMM_AENC_ProcessCachedFrame(SAMPLE_AENC_CTX_S *ctx, RK_S32 s32MilliSec);
```

【参数】

参数名	描述	输入/输出
ctx	音频编码句柄	输入
s32MilliSec	等待时间（毫秒）	输入

【返回值】

返回值	描述
int	0 表示成功，其他值表示失败

4.2.9 aoa_configs

成员名称	描述
mode	0:纯录音模式，1：语音唤醒模式
rate	采样率，只能使用16000
period_size	一帧的采样点个数
fw_path	模型路径，mode为1时必须填写

4.2.10 aoa_results

成员名称	描述
event	唤醒事件，当使用语音唤醒模式时有效
pts	音频对应的时间戳
seq_hw	dma 搬运序列号
seq_appl	接口调用序列号
seq_total	总搬运序列号

4.3 AOV配置

4.3.1 系统配置

4.3.1.1 Kernel

1. Config配置

RV1106:

arch/arm/configs/rv1106-pm.config

arch/arm/configs/rv1106-wakeup.config

RV1106B:

arch/arm/configs/rv1106b-pm.config

arch/arm/configs/rv1106b-wakeup.config

RV1126B

arch/arm/configs/rv1126b-pm.config

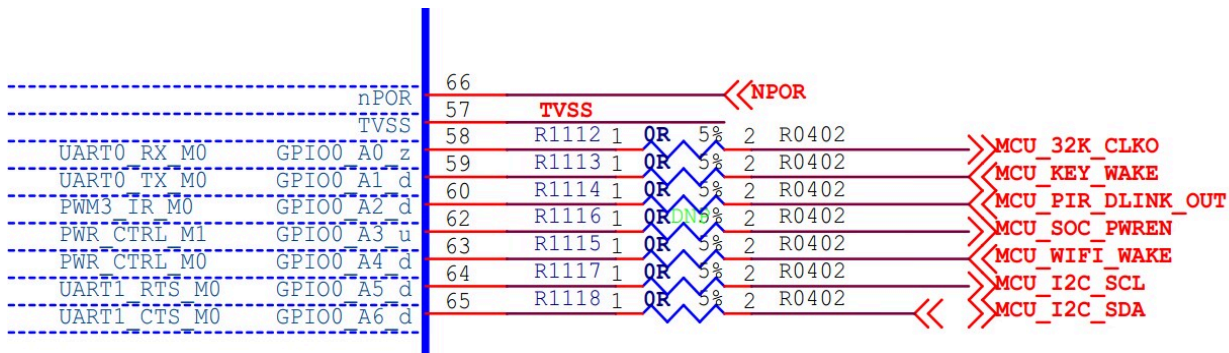
arch/arm/configs/rv1126b-aov.config

2. PM-GPIO配置

休眠后，只有PMUIO有电，其他常规IO都掉电了，所以需要根据在待机时外设需要保持的状态，将GPIO0A0~GPIO0A6配置为上拉或者下拉。若该GPIO口没有接外设，则配置为下拉，以下为EVB2-V13参考配置：

```
rockchip_suspend: rockchip-suspend {
    compatible = "rockchip,pm-rv1106";
    status = "okay";
    rockchip,sleep-io-config = <
        (0
        | RKPM_IO_CFG_IOMUX_GPIO
        | RKPM_IO_CFG_GPIO_DIR_INPUT
        | RKPM_IO_CFG_PULL_DOWN
        | RKPM_IO_CFG_ID(0)
        )
        (0
        | RKPM_IO_CFG_IOMUX_GPIO
        | RKPM_IO_CFG_GPIO_DIR_INPUT
        | RKPM_IO_CFG_PULL_DOWN
        | RKPM_IO_CFG_ID(1)
        )
        (0
        | RKPM_IO_CFG_IOMUX_GPIO
        | RKPM_IO_CFG_GPIO_DIR_INPUT
        | RKPM_IO_CFG_PULL_DOWN
        | RKPM_IO_CFG_ID(2)
        )
        (0
        | RKPM_IO_CFG_IOMUX_GPIO
        | RKPM_IO_CFG_GPIO_DIR_INPUT
        | RKPM_IO_CFG_PULL_UP
        | RKPM_IO_CFG_ID(3)
        )
        (0
        | RKPM_IO_CFG_IOMUX_GPIO
        | RKPM_IO_CFG_GPIO_DIR_INPUT
        | RKPM_IO_CFG_PULL_UP
        | RKPM_IO_CFG_ID(4)
        )
        (0
        | RKPM_IO_CFG_IOMUX_GPIO
        | RKPM_IO_CFG_GPIO_DIR_INPUT
        | RKPM_IO_CFG_PULL_NONE
        | RKPM_IO_CFG_ID(5)
        )
        (0
        | RKPM_IO_CFG_IOMUX_GPIO
        | RKPM_IO_CFG_GPIO_DIR_INPUT
        | RKPM_IO_CFG_PULL_NONE
        | RKPM_IO_CFG_ID(6)
        )
    )
}
```

```
>;
};
```



3. 常规IO

保活外设如果和常规IO有连接，这些外设的IO要切换到高阻，否则会漏电到相应电源域的电源上，造成异常。

4.3.1.2 BoardConfig

BoardConfig.mk 添加以下两个配置：

```
export RK_APP_TYPE=RK_SAMPLE_AOV
export RK_ENABLE_AOV=y
```

4.3.2 按键唤醒配置

1. BoardConfig.mk打开rv1106-pm.config、rv1106-wakeup.config配置以

project/cfg/BoardConfig_BatteryIPC/BoardConfig-SPI_NOR-NONE-RV1106_EVB2_V11-BAT_IPC_AOV.mk为例：

```
export RK_KERNEL_DEFCONFIG_FRAGMENT="rv1106-tb.config rv1106-battery-ipc.config rv1106-
pm.config rv1106-wakeup.config rv1106-sdiowifi.config"
```

2. 配置内核

使能GPIO按键唤醒（对应EVB2板子上的key2）。需要注意该按键的初始状态需要和PM-GPIO中的配置保持一致，否则可能会误唤醒。

```
diff --git a/arch/arm/boot/dts/rv1106g-evb2-v10.dts b/arch/arm/boot/dts/rv1106g-evb2-v10.dts
index 41e31555a203..f4e758927766 100644
--- a/arch/arm/boot/dts/rv1106g-evb2-v10.dts
+++ b/arch/arm/boot/dts/rv1106g-evb2-v10.dts
@@ -50,6 +50,29 @@ wireless_wlan: wireless-wlan {
    WIFI,host_wake_irq = <&gpio1 RK_PB0 GPIO_ACTIVE_HIGH>;
    status = "okay";
};
+
+ gpio-keys {
+     compatible = "gpio-keys";
+     autorepeat;
```



```

+         pinctrl-names = "default";
+         pinctrl-0 = <&pwr_key>;
+         power-key {
+             gpios = <&gpio0 RK_PA1 GPIO_ACTIVE_HIGH>;
+             linux,code = <KEY_POWER>;
+             label = "GPIO Key Power";
+             debounce-interval = <100>;
+             wakeup-source;
+             /* gpio-key,wakeup; */
+         };
+     };
+};
+
+&pinctrl {
+    buttons {
+        pwr_key:pwr-key {
+            rockchip,pins = <0 RK_PA1 RK_FUNC_GPIO &pcfg_pull_down>;
+        };
+    };
+};
diff --git a/arch/arm/configs/rv1106-battery-ipc.config b/arch/arm/configs/rv1106-battery-ipc.config
index c9f8f8803a3..07c6f00f7c5a 100644
--- a/arch/arm/configs/rv1106-battery-ipc.config
+++ b/arch/arm/configs/rv1106-battery-ipc.config
@@ -268,3 +267,6 @@ CONFIG_SPI_ROCKCHIP_SFC=y
# CONFIG_VIDEO_S5C73M3 is not set
CONFIG_ZLIB_DEFLATE=y
CONFIG_ZLIB_INFLATE=y
+CONFIG_INPUT=y
+CONFIG_INPUT_KEYBOARD=y
+CONFIG_KEYBOARD_GPIO=y

```

如果触发方式要改成单边沿，dts中power-key节点增加如下：

```

wakeup-event-action = <EV_ACT_ASSERTED>;    //上升沿触发
wakeup-event-action = <EV_ACT_DEASSERTED>;  //下降沿触发

```

4.3.3 摄像头驱动

AOV 摄像头支持Standby 和掉电两种方式，如果摄像头本身支持Standby低功耗，建议选择Standby方式，开发会更简单。具体开发详见Rockchip_Trouble_Shooting_Linux5.10_Camera_CN.pdf

4.3.3.1 Standby模式

以SC200AI为例，取流时进入正常模式，关流时进入Standby低功耗模式。摄像头有不同特性，最好跟原厂沟通确认清楚。

4.3.3.2 掉电模式

软件恢复摄像头状态，硬件上三路电压在休眠状态下无需供电，但是由于摄像头状态是由于软件恢复的，所以AOV整体工作时间占空比会更长，时长跟I2C写寄存器序列相关。

4.3.3.3 补光灯控制

灯的功耗是较高的，而AOV模式下摄像头并不是一直在工作的，为了将灯的工作占空比降低即降低功耗，需要由驱动精准的对灯的开关进行控制。

4.3.4 WiFi/4G开发说明

硬件要求：

- WiFi/4G模块：一个是可唤醒的WiFi/4G_WAKE_GPIO，另一个是普通的WiFi/4G_NORMAL_GPIO；
- RV1106：一个是可唤醒的RK_WAKE_GPIO，另一个是普通RK_NORMAL_GPIO；
- WiFi/4G建议使用低功耗的，内部支持独立的网络协议栈，可以独立连接外部服务器，支持休眠状态下WiFi/4G保持连接，且支持网络唤醒，比如WiFi：海思Hi3861L，4G模块：合宙Air780E；

唤醒方式：

- RK_NORMAL_GPIO 与 WiFi/4G_WAKE_GPIO相连：目的是使RK主控可以主动唤醒WiFi/4G模块；
- WiFi/4G_NORMAL_GPIO 与 RK_WAKE_GPIO相连：目的是使WiFi/4G模块可以主动唤醒RK主控；

使用场景：

1. WiFi/4G负责基本的数据通信，且支持主控休眠后，WiFi/4G独立能与外部服务器保持连接，并处于低功耗模式下，且能通过服务器远程唤醒WiFi/4G模块，或RK主控通过GPIO中断主动唤醒WiFi/4G模块；
2. 当主控休眠需要进入休眠前，需要做如下操作：
 - 先发送休眠指令（比如AT指令）给WiFi/4G模块，让其保持低功耗模式；
 - 如果需要网络唤醒，则还需要发送远程服务器地址/端口给WiFi/4G，让WiFi/4G连接到远端服务器；
 - 最后rmmod卸载掉WiFi/4G驱动，目的是为了在休眠唤醒期间，kernel无需再处理WiFi/4G的驱动以及设备枚举问题；
3. 主控进入1秒一次休眠循环中，此时有以下几种情况：
 - 主控被PIR或其它外设唤醒，但此时并不需要WiFi/4G通信，则无需任何操作WiFi/4G；
 - 主控被PIR或其它外设唤醒，且需要连接WiFi网络通信，此时RK主控需要通过**拉高/拉低连接到WiFi/4G模块的RK_NORMAL_GPIO来唤醒WiFi模块**，然后主控需要重新加载WiFi/4G的KO，并配置IP地址后就可以联网；（注意：由于WiFi/4G只是处于低功耗模式，网络一直是连着的，所以唤醒后能快速连接网络）
 - WiFi/4G被远端服务器唤醒，WiFi/4G通过**拉高/拉低连接到主控的WIFI/4G_NORMAL_GPIO来唤醒RK主控**，此时主控需要重新加载WiFi/4G KO，并配置IP地址后就可以联网；
4. 循环到第2步；

WiFi/4G 软件流程参考

从以下地址获取具体的流程及步骤：

FTP地址: <ftp://www.rockchip.com.cn>

注意: FTP软件要支持中文编码

账号名: rkwifi

密码: Cng9280H8t

/11-Linux平台/WIFI低功耗方案/AOV

4.3.5 休眠状态时钟源

在休眠模式下, 时钟源可以选择PVTM分频或RTC输出时钟。PVTM方案可能存在一定的误差, 该误差受温度、电压和频率等因素影响, 从而影响休眠时长。SDK默认采用PVTM方案。

RTC方案:

硬件:

需要使能RTC (仅适用于RV1106, RV1103无内置RTC) 具体参考硬件参考设计。

内核配置:

```
&rtc {
    rockchip,rtc-suspend-bypass;
    status = "okay";
};

&rockchip_suspend {
    rockchip,sleep-mode-config = <
        (0
        | RKPM_SLP_32K_EXT
        )
    >;
    status = "okay";
};
```

```
CONFIG_RTC_DRV_ROCKCHIP=y
```

4.3.6 基于eMMC的AOV开发说明

基于EMMC的AOV开发流程和SPI NOR类似。由于EMMC在休眠唤醒过程中恢复时间长(约1~2秒), 严重影响了AOV运行功耗。故在开发AOV应用时, 需把EMMC当作SD卡一样使用, 在进入单帧模式时unbind设备驱动, 恢复到多帧模式时再bind设备驱动, 以减少EMMC驱动休眠唤醒的时间。此外由于需要把运行时所需的kernel、应用以及依赖库打包到ramfs上面, 所以相较于SPI NOR需要占用更多内存, 在打包文件系统时应尽量缩减ramfs镜像的大小。

4.3.6.1 板级配置

板级配置可以参考project/cfg/BoardConfig_IPC/BoardConfig-EMMC-NONE-RV1106_EVB2_V12-IPC_AOV.mk, 需要将kernel和应用、库文件都打包到boot分区, 并将其设置为initramfs(可读写)或者romfs(只读), 使其可以常驻内存。然后其他文件如iq文件, AI模型文件等可以打包到userdata分区。

```

export
RK_PARTITION_CMD_IN_ENV="32K(env),512K@32K(idblock),256K(uboot),32M(boot),1G(userdata),-(media)"
# boot可以选择initramfs或者romfs, boot分区常驻内存
export RK_PARTITION_FS_TYPE_CFG=boot@IGNORE@initramfs,userdata@/userdata@ext4
# 注释掉该配置, 使oem打包到boot分区中
# export RK_BUILD_APP_TO_OEM_PARTITION=y

```

内核需要添加编译配置:

```

diff --git a/arch/arm/configs/rv1106-wakeup.config b/arch/arm/configs/rv1106-wakeup.config
index 605fe719b696..e060517a7492 100644
--- a/arch/arm/configs/rv1106-wakeup.config
+++ b/arch/arm/configs/rv1106-wakeup.config
@@ -1,3 +1,4 @@
+CONFIG_BLK_DEV_INITRD=y
# CONFIG_ETHERNET is not set
CONFIG_HIGH_RES_TIMERS=y
CONFIG_INPUT=y

```

4.3.6.2 应用流程

应用流程可参考sample_aov_vi.c。在进入单帧模式前, 需要先卸载ext4文件系统, 然后解绑eMMC设备驱动。在进行了一系列操作后, ext4文件系统将不能访问, 系统会运行在ramfs之上。恢复到多帧模式时, 则进行对应的恢复操作。

```

static void enter_single_frame_mode(RK_S32 s32DevId) {
    RK_S32 ret = RK_SUCCESS;
    if (g_bEnableEmmc) {
        // 0. sync filesystem
        sync();
        // 1. umount filesystem
        ret = umount2("/userdata", MNT_DETACH);
        if (ret != RK_SUCCESS)
            RK_LOGE("umount /userdata failed %s", strerror(errno));
        // 2. unbind emmc device
        ret = SAMPLE_COMM_AOV_UnbindEmmc();
        if (ret != RK_SUCCESS)
            RK_LOGE("unbind emmc failed %s", strerror(errno));
    }
    // 3. enter single frame mode
    // When booting, the default is multi-frame mode, so you need to pause the
    // stream and pick up the remaining frames
    SAMPLE_COMM_ISP_SingleFrame(s32DevId);
    SAMPLE_COMM_AOV_DisableNonBootCPUs();
}

```

在恢复到多帧模式前, 首先绑定设备驱动, 并且等待eMMC驱动完成probe操作, 该步骤已经打包在SAMPLE_COMM_AOV_BindEmmc函数中。然后需要挂载此前卸载掉的ext4文件系统。

```
static void enter_multi_frame_mode(RK_S32 s32DevId) {
    RK_S32 ret = RK_SUCCESS;
    // 1. enter multi frame mode
    SAMPLE_COMM_ISP_MultiFrame(s32DevId);
    SAMPLE_COMM_AOV_EnableNonBootCPUs();
    if (g_bEnableEmmc) {
        // 2. bind emmc device
        ret = SAMPLE_COMM_AOV_BindEmmc();
        if (ret != RK_SUCCESS)
            RK_LOGE("bind emmc failed %s", strerror(errno));
        // 3. mount filesystem
        ret = mount("/dev/mmcblk0p5", "/userdata", "ext4", 0, NULL);
        if (ret != RK_SUCCESS)
            RK_LOGE("mount /userdata failed %s", strerror(errno));
    }
}
```

在 mount 函数中，传入的分区名字与BoardConfig中的 RK_PARTITION_CMD_IN_ENV 参数相关，编译脚本会根据该参数自动生成分区信息。如参考配置BoardConfig-EMMC-NONE-RV1106_EVB2_V12-IPC_AOV.mk中，userdata分区排在第5，对应的是设备名字是 /dev/mmcblk0p5，以此类推。

```
export
RK_PARTITION_CMD_IN_ENV="32K(env),512K@32K(idblock),256K(uboot),32M(boot),1G(userdata),-
(media)"
```

4.4 AOA配置

4.4.1 板级配置

板级配置可以参考project/cfg/BoardConfig_BatteryIPC/BoardConfig-EMMC-NONE-RV1126B_EVB2_V12-AOV.mk，需要添加内核配置 arch/arm/configs/rv1126b-aoa.config

```
# Kernel defconfig fragment
export RK_KERNEL_DEFCONFIG_FRAGMENT="rv1126b-ipc.config rv1126b-pm.config rv1126b-aoa.config
rv1126b-aoa.config"

# Kernel dts
export RK_KERNEL_DTS=rv1126b-evb2-v12.dts
```

4.4.2 应用流程

应用流程可参考sample_aov_vi_iva_venc.c。

1. 音频初始化：使能并启动aoa模块，进入aov 模式时不要解绑音频驱动

```
memset(&config, 0, sizeof(config));
config.mode = 0;
```

```

config.rate = 16000;
config.period_size = 256;

hdl_aoa = aoa_create(&config);
if (!hdl_aoa) {
    fprintf(stderr, "ERR: create aoa failed\n");
    goto __clean_aobuf;
}

ret = aoa_start(hdl_aoa);
if (ret < 0) {
    fprintf(stderr, "ERR: aoa start failed: %d\n", ret);
    goto __clean_aenc;
}

```

2. 读取音频帧：唤醒后读出音频帧

```

int read_size = aoa_read(hdl_aoa, aoa_buf, config.period_size, &res, -1);
if (read_size <= 0) {
    fprintf(stderr, "ERR: aoa read failed: %d\n", read_size);
    continue;
}

```

3. 数据处理：编码数据可以参考以下流程，添加缓存机制，保障平稳运行

```

ret = cache_audio_frm(aenc, blk, pcm_frame_size, res.pts, seq);
if (ret != RK_SUCCESS) {
    fprintf(stderr, "ERR: cache audio frame failed: %d\n", ret);
    continue;
}
ret = SAMPLE_COMM_AENC_ProcessCachedFrame(&aenc, 10);
if (ret != RK_SUCCESS) {
    RK_LOGE("SAMPLE_COMM_AENC_ProcessCachedFrame %#X", ret);
}

```

AOV模式下，可以通过下面接口增加使用的核心数，加快应用恢复速度，延长编码器运行时间而不增加唤醒时间

```

SAMPLE_COMM_AOV_DisableNonBootCPUs()
SAMPLE_COMM_AOV_EnableNonBootCPUs()

```

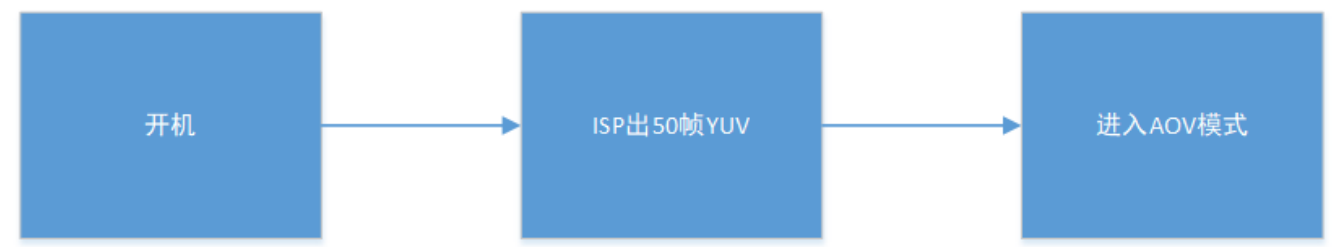
5. 调试说明

5.1 Demo说明

5.1.1 ISP 取流测试

说明：

从ISP取一帧，并进入休眠



代码路径：

project/app/aov_sample/sample_aov_vi.c

快速使用：

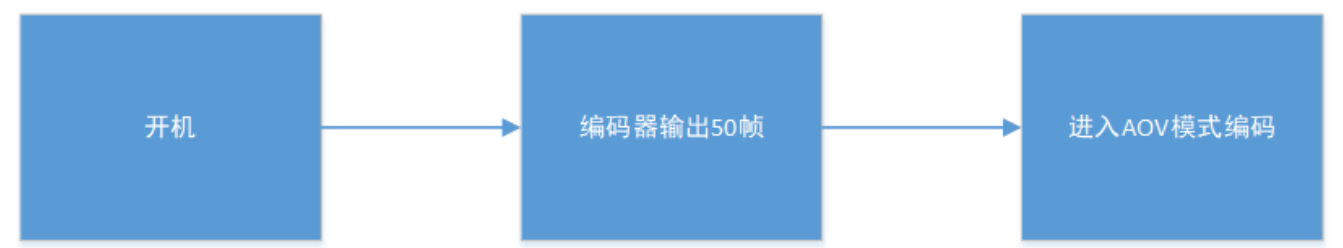
sample_aov_vi -w 1920 -h 1080 -a /etc/iqfiles/ --aov_loop_count 50 --boot_frame 50 --suspend_time 1000

选项	描述	默认值
-w	分辨率宽	1920
-h	分辨率高	1080
-a --aiq	内置ISP功能启用， 输入该选项启用内置ISP功能， 无参数则使用默认值， 参数为aiq文件所在文件夹路径。	NULL
--suspend_time	系统休眠时长，单位ms	1000
--aov_loop_count	设置AOV输出多少帧之后退出程序	-1
--boot_frame	设置运行程序后多少帧之后进入AOV模式	60
--output_path	yuv的输出路径，文件名为vi_0.bin	NULL
--vi_frame_mode	单帧或者单帧多帧模式切换。 0：单帧模式， 1：单帧->多帧->单帧	0

5.1.2 主码流编码测试

说明

从ISP取一帧并送编码，编码完成之后进入休眠，如果有插sd卡，会将码流存储到/mnt/sdcard/wakeup_frame_%d，阈值为10M，如果超过100个文件夹，需要手动删除。



代码路径：

project/app/aov_sample/sample_aov_vi_venc.c

快速使用：

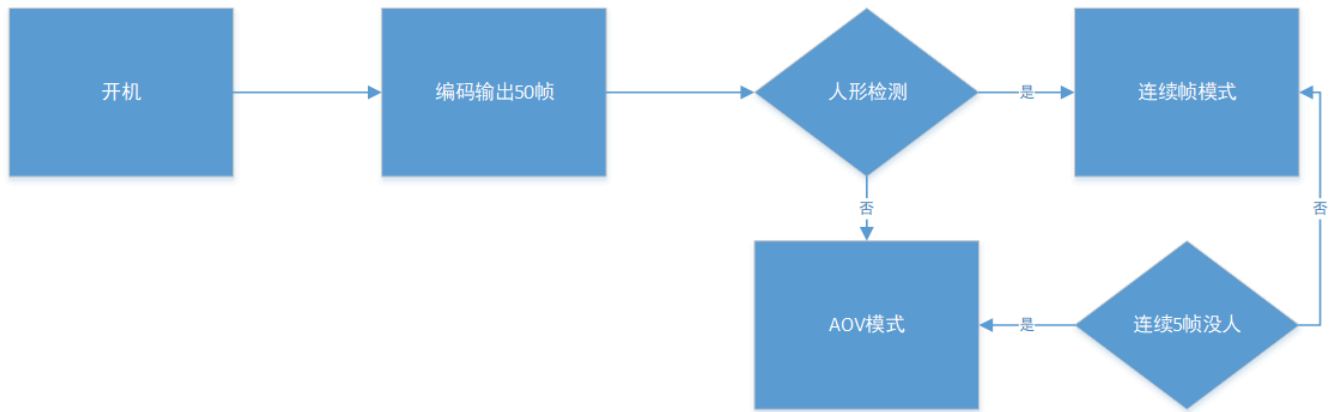
sample_aov_vi_venc -w 1920 -h 1080 -a /etc/iqfiles/

选项	描述	默认值
-w	分辨率宽	1920
-h	分辨率高	1080
-a --aiq	内置ISP功能启用， 输入该选项启用内置ISP功能，无参数则使用默认值， 参数为aiq文件所在文件夹路径。	NULL
-e --encode	设置编码方式	h264cbr
-b --bitrate	设置编码码率	4096
-r --wrap	卷绕开关，0： 关闭，1： 开启	0
--wrap_lines	卷绕行设置，0： height/2， 1： height/4， 2： height/8	1
--suspend_time	系统休眠时长， 单位ms	1000
--aov_loop_count	设置AOV输出多少帧之后退出程序	-1
--boot_frame	设置运行程序后多少帧之后进入AOV模式	60

5.1.3 人形检测测试

说明：

从ISP 取流一帧，送NN做人形识别，如果识别到人形会切换为连续帧模式，等到连续5帧没识别到人形，再进入休眠。



代码路径：

```
project/app/aov_sample/sample_aov_vi_iva_venc.c
```

快速使用：

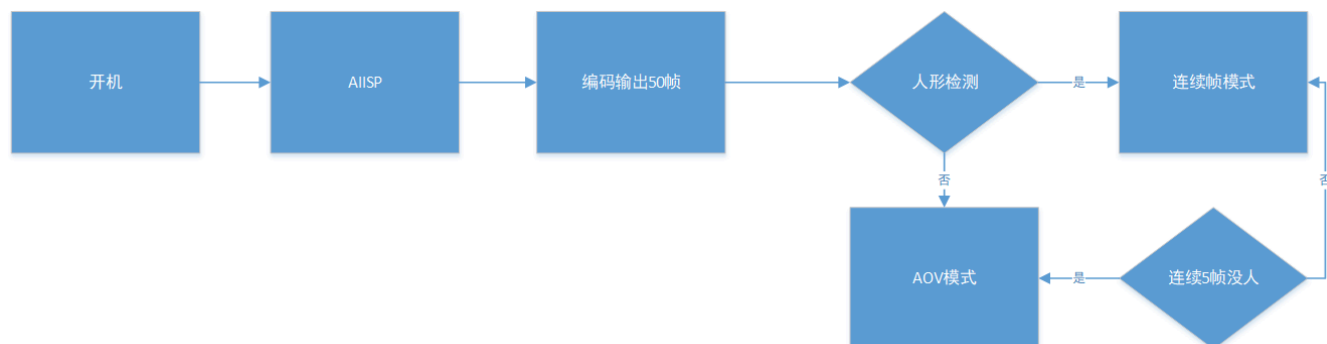
```
sample_aov_vi_iva_venc -w 1920 -h 1080 -a /etc/iqfiles/
```

选项	描述	默认值
-w	分辨率宽	1920
-h	分辨率高	1080
-a --aiq	内置ISP功能启用， 输入该选项启用内置ISP功能， 无参数则使用默认值， 参数为aiq文件所在文件夹路径。	NULL
-e --encode	设置编码方式	h265vbr
-b --bitrate	设置编码码率	4096
-r --wrap	卷绕开关，0： 关闭，1： 开启	0
-f --fps	设置摄像头帧率	25
--wrap_lines	卷绕行设置，0： height/2， 1： height/4， 2： height/8	1
--suspend_time	系统休眠时长， 单位ms	1000
--aov_loop_count	设置AOV输出多少帧之后退出程序	-1
--boot_frame	设置运行程序后多少帧之后进入AOV模式	60
--enable_multi_frame	识别到人形是否进入连续帧，1： 进入连续帧，0： 继续休眠	1
--low_power	低功耗模式，0： 关闭，1： 开启	0
--camera_mirror_flip	设置摄像头旋转，0: 0, 1: 90, 2: 180, 3: 270	0
--fast_ae	使能曝光快速收敛，0： 关闭，1： 开启	0
--enable_save_mp4	使能保存MP4，0： 关闭，1： 开启， <i>同时可以开启连续录音</i>	1

5.1.4 AIISP测试

说明：

经过AIISP处理后，送NN做人形识别，如果识别到人形会切换为连续帧模式，等到连续5帧没识别到人形，再进入休眠。



代码路径：

project/app/aov_sample/sample_aov_aiisp_venc.c

快速使用：

sample_aov_aiisp_iva_venc -w 1920 -h 1080 -a /etc/iqfiles/ -I 0 -e h264cbr

选项	描述	默认值
-w	分辨率宽	1920
-h	分辨率高	1080
-a --aiq	内置ISP功能启用，输入该选项启用内置ISP功能，无参数则使用默认值，参数为aiq文件所在文件夹路径。	NULL
-e --encode	设置编码方式	h265vbr
-b --bitrate	设置编码码率	4096
-r --wrap	卷绕开关，0：关闭，1：开启	0
-f --fps	设置摄像头帧率	25
--wrap_lines	卷绕行设置，0：height/2，1：height/4，2：height/8	1
--suspend_time	系统休眠时长，单位ms	1000
--aov_loop_count	设置AOV输出多少帧之后退出程序	-1
--boot_frame	设置运行程序后多少帧之后进入AOV模式	60
--enable_multi_frame	识别到人形是否进入连续帧，0：继续休眠，1：进入连续帧	1
--enable_aiisp	是否开启AIISP，0：关闭，1：开启	
--enable_aov	是否开启AOV模式，0：关闭，1：开启。 如果关闭可以用rtsp预览码流rtsp://xx.xx.xx.xx/live/0	1
--low_power	低功耗模式，0：关闭，1：开启	1
--camera_mirror_flip	设置摄像头旋转，0: 0, 1: 90, 2: 180, 3: 270	0
--fast_ae	使能曝光快速收敛，0：关闭，1：开启	0

5.1.5 双摄ISP取流测试

说明：

从ISP0与ISP1各取一帧，并进入休眠：



代码路径：

project/app/aov_sample/sample_aov_multi_vi.c

快速使用：

```
sample_aov_multi_vi -s 0 -w 2048 -h 1536 -s 1 -w 1920 -h 1080 -a /etc/iqfiles/ --aov_loop_count 10
```

选项	描述	默认值
-s	0表示主摄，1表示副摄，用于设置对应摄像头的分辨率	0
-w	分辨率宽	1920
-h	分辨率高	1080
-a --aiq	内置ISP功能启用，输入该选项启用内置ISP功能，无参数则使用默认值，参数为aiq文件所在文件夹路径。	NULL
--suspend_time	系统休眠时长，单位ms	1000
--aov_loop_count	设置AOV输出多少帧之后退出程序	-1
--boot_frame	设置运行程序后多少帧之后进入AOV模式	60
--vi_frame_mode	单帧或者单帧多帧模式切换。 0：单帧模式，1：单帧->多帧->单帧	0
--output_path	yuv的输出路径，文件名为vi_0.bin	NULL
--isp_mode	isp工作模式，0：回读，1：半直通，2：全直通。回读模式下两路isp都需要raw buffer，半直通只需要配置一路raw buffer，全直通不需要raw buffer。但是相对的，全直通，半直通比起回读模式，对于双目sensor之间的曝光同步以及vblank具有更严格的要求，不满足要求会发生丢帧。所以从内存和效果综合考虑，默认使用半直通	1
--cif_buff_cnt	驱动raw buffer的数量，对于回读和半直通有效，如果isp帧率不足，可以适当增加该buffer数量	1
--enable_change_resolution	使能切换大小分辨率。在进入AOV模式时，切换sensor和编码器输出小图，以减少功耗；切换到连续帧模式时输出大图	0
--enable_scale	在使能切换大小分辨率功能时，不取切换后端的编码器分辨率，而是由isp缩放大图送给后端编码，可以减少应用的复杂度，但相对的AOV模式下功耗会上升	0

5.1.6 双摄主码流编码测试

说明：

从ISP0与ISP1取一帧并送编码，全部编码完成之后进入休眠，如果有插sd卡会将码流存储到/mnt/sdcard/wakeup_frame_%d，阈值为10M，如果超过100个文件夹，需要手动删除。



代码路径：

project/app/aov_sample/sample_aov_multi_vi_venc.c

快速使用：

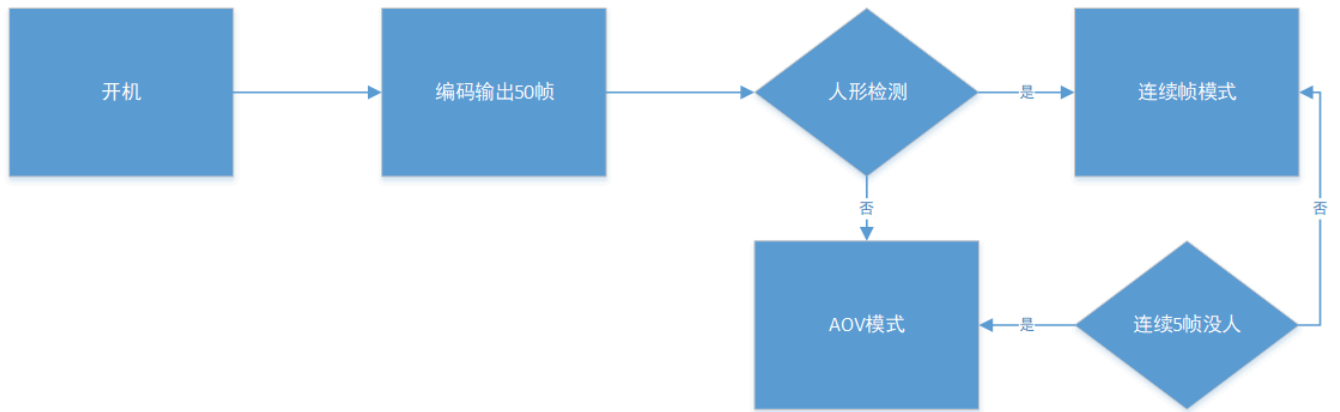
sample_aov_multi_vi_venc -s 0 -w 2048 -h 1536 -s 1 -w 1920 -h 1080 -a /etc/iqfiles/ --aov_loop_count 10

选项	描述	默认值
-s	0表示主摄，1表示副摄，用于设置对应摄像头的分辨率	0
-w	分辨率宽	1920
-h	分辨率高	1080
-a --aiq	内置ISP功能启用， 输入该选项启用内置ISP功能， 无参数则使用默认值， 参数为aiq文件所在文件夹路径。	NULL
-e --encode	设置编码方式	h264vbr
-b --bitrate	设置编码码率	4096
-r --wrap	卷绕开关，0： 关闭，1： 开启	0
--wrap_lines	卷绕行设置，0： height/2， 1： height/4， 2： height/8	1
--suspend_time	系统休眠时长， 单位ms	1000
--aov_loop_count	设置AOV输出多少帧之后退出程序	-1
--boot_frame	设置运行程序后多少帧之后进入AOV模式	60
--enable_save_sdcard	保存码流到sd卡	0

5.1.7 双摄人形检测测试

说明：

从ISP0与ISP1各自送帧到编码器。异步线程取小图，送到IVS做动静判决，再送NN做人形识别，如果识别到人形会切换为连续帧模式，等到连续5秒没识别到人形，再进入休眠。



project/app/aov_sample/sample_aov_multi_vi_iva_venc.c

快速使用：

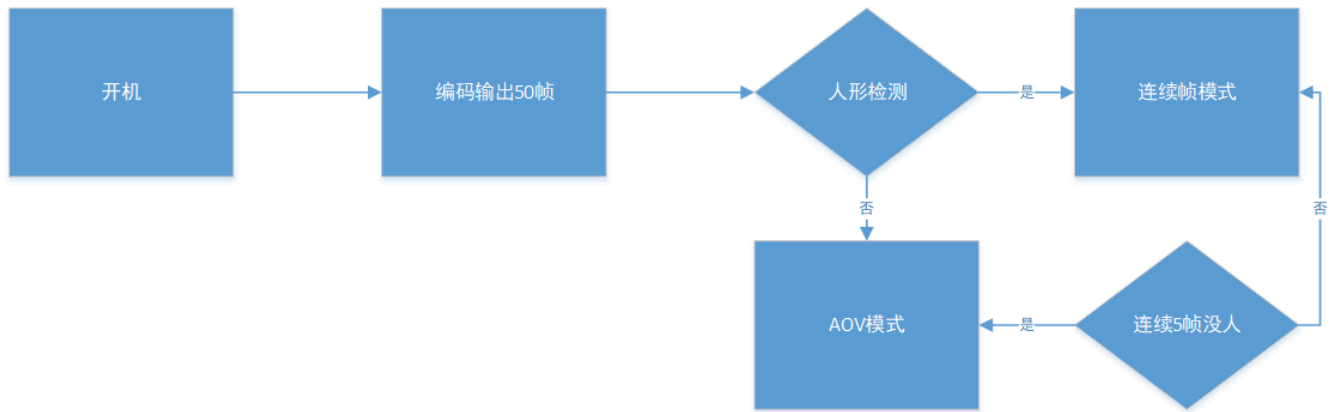
sample_aov_multi_vi_iva_venc -s 0 -w 2048 -h 1536 -s 1 -w 1920 -h 1080 -a /etc/irqfiles/

选项	描述	默认值
-s	0表示主摄，1表示副摄，用于设置对应摄像头的分辨率	0
-w	分辨率宽	1920
-h	分辨率高	1080
-a --aiq	内置ISP功能启用，输入该选项启用内置ISP功能，无参数则使用默认值，参数为aiq文件所在文件夹路径。	NULL
-e --encode	设置编码方式	h264vbr
-b --bitrate	设置编码码率	4096
-r --wrap	卷绕开关，0：关闭，1：开启	0
--wrap_lines	卷绕行设置，0：height/2，1：height/4，2：height/8	1
--suspend_time	系统休眠时长，单位ms	1000
--aov_loop_count	设置AOV输出多少帧之后退出程序	-1
--boot_frame	设置运行程序后多少帧之后进入AOV模式	60
--isp_mode	isp工作模式，0：回读，1：半直通，2：全直通。回读模式下两路isp都需要raw buffer，半直通只需要配置一路raw buffer，全直通不需要raw buffer。但是相对的，全直通，半直通比起回读模式，对于双目sensor之间的曝光同步以及vblank具有更严格的要求，不满足要求会发生丢帧。所以从内存和效果综合考虑，默认使用半直通	1
--cif_buff_cnt	驱动raw buffer的数量，对于回读和半直通有效，如果isp帧率不足，可以适当增加该buffer数量	1
--vi_buff_cnt	isp输出buffer的数量，如果isp帧率不足，可以适当增加该buffer数量	1
--enable_change_resolution	使能切换大小分辨率。在进入AOV模式时，切换sensor和编码器输出小图，以减少功耗；切换到连续帧模式时输出大图	0
--enable_scale	在使能切换大小分辨率功能时，不取切换后端的编码器分辨率，而是由isp缩放大图送给后端编码，可以减少应用的复杂度，但相对的AOV模式下功耗会上升	0
--enable_save_sdcard	保存码流到sd卡	0

5.1.8 双摄上下非融合拼接测试

说明：

从ISP0与ISP1各取一帧，由ISP做上下拼接，再将拼接后的结果送给编码器。异步线程取小图，送到IVS做动静判决，再送NN做人形识别，如果识别到人形会切换为连续帧模式，等到连续5秒没识别到人形，再进入休眠。



```
project/app/aov_sample/sample_aov_multi_vi_nonblend_venc.c
```

快速使用：

```
sample_aov_multi_vi_nonblend_venc -w 1920 -h 1080 -a /etc/iqfiles/
```

选项	描述	默认值
-w	分辨率宽	1920
-h	分辨率高	1080
-a --aiq	内置ISP功能启用， 输入该选项启用内置ISP功能， 无参数则使用默认值， 参数为aiq文件所在文件夹路径。	NULL
-e --encode	设置编码方式	h264vbr
-b --bitrate	设置编码码率	4096
-r --wrap	卷绕开关，0： 关闭，1： 开启	0
--wrap_lines	卷绕行设置，0： height/2， 1： height/4， 2： height/8	1
--suspend_time	系统休眠时长，单位ms	1000
--aov_loop_count	设置AOV输出多少帧之后退出程序	-1
--boot_frame	设置运行程序后多少帧之后进入AOV模式	60
--isp_mode	isp工作模式，0： 回读，1： 半直通，2： 全直通。回读模式下两路isp都需要raw buffer，半直通只需要配置一路raw buffer，全直通不需要raw buffer。但是相对的，全直通，半直通比起回读模式，对于双目sensor之间的曝光同步以及vblank具有更严格的要求，不满足要求会发生丢帧。所以从内存和效果综合考虑，默认使用半直通	1
--cif_buff_cnt	驱动raw buffer的数量，对于回读和半直通有效，如果isp帧率不足，可以适当增加该buffer数量	1
--vi_buff_cnt	isp输出buffer的数量，如果isp帧率不足，可以适当增加该buffer数量	1
--enable_save_sdcard	保存码流到sd卡	0

5.2 休眠唤醒耗时排查

5.2.1 AOV耗时确认

方法一：power_en管脚接出来，通过示波器看工作时长；

方法二：摄像头对着秒表计时器，查看帧与帧的平均间隔。

5.2.2 软件耗时排查

1. 关闭串口异步打印：

```
关闭选项CONFIG_RK_CONSOLE_THREAD
```

2. 打开PM-Log，检查整体驱动休眠唤醒耗时，如果有发现耗时偏长的驱动，参考sd卡驱动让其不参与AOV休眠唤醒过程。

```
echo 0 > /sys/power/pm_async
echo N > /sys/module/printk/parameters/console_suspend
echo 1 > sys/power/pm_print_times
```

5.3 排查音频卡顿掉帧

1. 休眠前，比较 seq_appl 和 seq_hw，相等就说明休眠期间的音频数据都已经读完了。

```
read_size = aoa_read(hdl_aoa, pcm_buf, period_size, &res, read_timeout_ms);
// seq_appl:应用调用 aoa_read 的次数
// res.seq_hw:DMA搬运次数
// 唤醒后, seq_hw 会一次性增加休眠期间搬运的帧数, 而 seq_appl 随每次 aoa_read 调用逐步增加。
// 当两者相等, 说明休眠期间的所有音频数据都已读取完毕
printf("seq_appl=%u, seq_hw=%u\n", res.seq_appl, res.seq_hw);
```

2. 检查任意两次调用aoa_read的时间是否超过了1.8s，尤其应该注意休眠前后的时间。

```
clock_gettime(CLOCK_BOOTTIME, &t);
aoa_read(hdl_aoa, pcm_buf, period_size, &res, read_timeout_ms);
```

3. 检查是否已经无法分配缓存

```
blk = RK_MPI_MB_GetMB(aenc.pool_id, pcm_frame_size, RK_TRUE);
if (!blk) {
    fprintf(stderr, "ERR: get mb block failed\n");
    continue;
}
```

这种情况通常是由于编码器性能不足，无法及时编码所有音频帧，导致缓存被耗尽。可以依次采用以下策略进行优化：

降低编码器码率，最低可设置为 32kbps。

```
diff --git a/common/sample_comm_aenc.c b/common/sample_comm_aenc.c
index 69dfc77..0af8764 100644
--- a/common/sample_comm_aenc.c
+++ b/common/sample_comm_aenc.c
@@ -210,7 +210,7 @@ RK_S32 SAMPLE_COMM_AENC_SetAttr(SAMPLE_AENC_CTX_S *ctx) {
    pstChnAttr->stCodecAttr.pstResv = RK_NULL;
    if (pstChnAttr->stCodecAttr.enType == RK_AUDIO_ID_MP3) {
        pstChnAttr->stCodecAttr.u32Resv[0] = 1152;
-       pstChnAttr->stCodecAttr.u32Resv[1] = 64000;
+       pstChnAttr->stCodecAttr.u32Resv[1] = 32000;
    }
```

提高 CPU 频率，以增强系统处理能力。

```
cat /sys/devices/system/cpu/cpufreq/policy0/scaling_available_governors
echo userspace > /sys/devices/system/cpu/cpufreq/policy0/scaling_governor
cat /sys/devices/system/cpu/cpufreq/policy0/scaling_available_frequencies
echo 1416000 > /sys/devices/system/cpu/cpufreq/policy0/scaling_setspeed
```

减少休眠时间，可通过调整 `SAMPLE_COMM_AOV_SetPeriodTime` 实现。

5.4 排查视频不出流问题

1. Camera驱动确认是否有调用到流恢复：

```
case RKMODULE_SET_QUICK_STREAM:
    stream = *((u32 *)arg);
    if (stream)
        ret = sc200ai_write_reg(sc200ai->client, SC200AI_REG_CTRL_MODE,
                                SC200AI_REG_VALUE_08BIT, SC200AI_MODE_STREAMING);
    else
        ret = sc200ai_write_reg(sc200ai->client, SC200AI_REG_CTRL_MODE,
                                SC200AI_REG_VALUE_08BIT, SC200AI_MODE_SW_STANDBY);
```

如果未增加log，也可以读取Sensor的出流寄存器判断是否正常工作

```
i2ctransfer -f -y 4 w2@0x30 0x01 0x00 r1
```

2. 查看cif、isp节点是否有出流：

```
cat /proc/rkcif-mipi-lvds
cat /proc/rkisp-vir0
```

3. AOV模式下整条媒体通路均不能使用丢帧策略

4. 卷绕情况下不出流，可以依次使用以下三个策略：

将卷绕行改大

可以将卷绕行改成1/2，或者全BUF

提升编码器频率

```
sysdrv/source/kernel$ git diff arch/arm/boot/dts/rv1106.dtsi
diff --git a/arch/arm/boot/dts/rv1106.dtsi b/arch/arm/boot/dts/rv1106.dtsi
index 6180d18da..6830e75f1 100644
--- a/arch/arm/boot/dts/rv1106.dtsi
+++ b/arch/arm/boot/dts/rv1106.dtsi
@@ -76,7 +76,7 @@ cpu_pvtpll: cpu-pvtpll {

        rkvinc_pvtpll: pvtpll-0 {
                compatible = "fixed-clock";
-               clock-frequency = <410000000>;
+               clock-frequency = <500000000>;
                clock-output-names = "clk_pvtpll_0";
                #clock-cells = <0>;
        };
@@ -1272,9 +1272,9 @@ rkvinc: rkvinc@ffa50000 {
        interrupt-names = "irq_rkvinc";
        clocks = <&cru ACLK_VEPU>, <&cru HCLK_VEPU>, <&cru CLK_CORE_VEPU>;
        clock-names = "aclk_vcodec", "hclk_vcodec", "clk_core";
-       rockchip,normal-rates = <300000000>, <0>, <410000000>;
+       rockchip,normal-rates = <300000000>, <0>, <500000000>;
        assigned-clocks = <&cru ACLK_VEPU>, <&cru CLK_CORE_VEPU>;
-       assigned-clock-rates = <300000000>, <400000000>;
+       assigned-clock-rates = <300000000>, <500000000>;
        resets = <&cru SRST_A_VEPU>, <&cru SRST_H_VEPU>, <&cru SRST_CORE_VEPU>;
        reset-names = "video_a", "video_h", "video_core";
        rockchip,srv = <&mpv_srv>;
```

dts修改完，需要输这个指令才会生效

```
echo clk_core_vepu 500000000 > /proc/clk/rate
```

降低ISP频率

```
diff --git a/drivers/media/platform/rockchip/isp/hw.c
b/drivers/media/platform/rockchip/isp/hw.c
index 6457772f7b89..aaa1647b309f 100644
--- a/drivers/media/platform/rockchip/isp/hw.c
+++ b/drivers/media/platform/rockchip/isp/hw.c
@@ -683,10 +683,10 @@ static const struct isp_clk_info rv1106_isp_clk_rate[] = {
        .clk_rate = 200,
        .refer_data = 2688,
    }, {
-       .clk_rate = 350,
+       .clk_rate = 200,
        .refer_data = 3072,
```

```
    }, {  
-        .clk_rate = 440,  
+        .clk_rate = 200,  
        .refer_data = 3840,  
    }  
};
```

6. 典型功耗数据

6.1 基础休眠功耗

6.1.1 RV1106

主控端不含外设休眠时整机功耗应该在5mW左右，如测试数据不符合，应该检查是否漏电到主控端，参考章节PM-GPIO配置，详细如下

1. 测试命令

```
io -4 0xff300048 320000000  
echo mem > /sys/power/state
```

2. 功耗数据

电路	电流
3V3_STB	约0.05mA
0V9_STB	约0.15mA(RTC)\约0.41mA(PVTM)
DDR_1V36	约1.8mA

6.1.2 RV1106B

主控端不含外设休眠时整机功耗应该在4.1mW左右，如测试数据不符合，应该检查是否漏电到主控端，参考章节PM-GPIO配置，详细如下

1. 测试命令

```
io -4 0x20514094 320000000  
echo mem > /sys/power/state  
echo mem > /sys/power/state #需要输入两次，确保机器在休眠状态
```

2. 功耗数据

电路	电流
3V3_STB	约0.1mA
DDR_1V36	约1.8mA

6.2 RV1106 SC200AI数据

6.2.1 编码一帧+人形检测

1. 测试命令

```
sample_aov_vi_iva_venc -w 1920 -h 1080 -a /etc/iqfiles/ --enable_multi_frame 0 --suspend_time 1000 --low_power 1
```

- --enable_multi_frame 0：表示识别到人形不会进入连续帧模式，1：表示识别到人形会进入连续帧模式
- --suspend_time 1000，指定休眠时间，单位ms
- --low_power，使能低功耗模式

2. 功耗实测数据

休眠时长	工作时长	整机功耗
1000ms	75ms	39mW
500ms	75ms	66mW
330ms	75ms	92mW

6.2.2 AIISP+编码一帧+人形检测

1. 测试命令

```
sample_aov_aiisp_iva_venc -w 1920 -h 1080 -a /etc/iqfiles/ --suspend_time 1000 --enable_multi_frame 0
```

- --enable_multi_frame 0：表示识别到人形不会进入连续帧模式，1：表示识别到人形会进入连续帧模式
- --suspend_time 1000，指定休眠时间，单位ms

2. 功耗实测数据（AIISP 使能且工作情况下）

休眠时长	工作时长	整机功耗
1000ms	75ms	76mW
500ms	75ms	129mW
330ms	75ms	170mW

6.3 RV1106B SC450AI数据

6.3.1 编码一帧 + 人形检测

1. 测试命令

```
sample_aov_vi_iva_venc -w 2688 -h 1520 -a /etc/iqfiles/ --enable_save_mp4 1 --enable_bypass 1 -r 1 --aov_loop_count -1 --camera_mirror_flip 0 --suspend_time 1000
```

- --suspend_time 1000，指定休眠时间，单位ms

2. 功耗实测数据

出帧间隔	工作时长	整机功耗
1000ms	66ms	34mW
2000ms	66ms	19.3mW
3000ms	66ms	14.1mW

6.4 RV1106B 双目SC200AI数据

6.4.1 非融合拼接+编码一帧+人形检测

1. 测试命令

```
sample_aov_multi_vi_nonblend_stitch_venc --suspend_time 1000 &
```

- --suspend_time 1000，指定休眠时间，单位ms

2. 功耗实测数据

出帧间隔	工作时长	整机功耗
1000ms	98ms	55.7mW
2000ms	98ms	30.8mW
3000ms	98ms	22.6mW

7. 常见问题

7.1 应用层时间怎么加上休眠时间

```
struct timespec t;  
clock_gettime(CLOCK_BOOTTIME, &t);
```

7.2 如何获取唤醒源

1. 如果是注册成按键事件，可以参考以下接口

```
bool SAMPLE_COMM_AOV_GetGpioIrqStat();
```

2. 可以通过以下节点查阅

```
cat sys/kernel/wakeup_reasons/last_resume_reason
```

7.3 为何AOV SDK开机后SD 卡无法识别

SDK默认将SD卡驱动卸载掉，可以使用如下命令将SD卡加载回来

```
# rv1106  
echo "ffaa0000.mmc" > /sys/bus/platform/drivers/dwmmc_rockchip/bind  
# rv1106b  
echo "20d20000.mmc" > /sys/bus/platform/drivers/dwmmc_rockchip/unbind
```

7.4 如何解决AOV过程中，SD卡或者4G传输数据时短暂没数据

SDK提供如下接口，用于在单帧模式下可以由应用调用再手动出一帧，每调用第一次出一帧：

```
RK_MPI_VI_DevEnableSingle1Frame
```

7.5 如何更低功耗控制红外灯

在AOV过程中，红外灯不需要常亮，可以通过如下接口控制红外补光灯的开关

```
RK_MPI_VI_DevEnableLight
```

注意：接口使能红外灯，这个方式不能再使用export 等其他外部方式控制红外灯。摄像头驱动开发参考**补光灯控制**章节。